

1. Procedimiento básico con parámetro IN

Puede caer: mostrar productos de una categoría

```
CREATE OR REPLACE PROCEDURE mostrar_productos_categoria
(p_categoria IN VARCHAR2)
IS
BEGIN
    FOR r IN (
        SELECT NOMBRE, PRECIO
        FROM PRODUCTOS
        WHERE CATEGORIA = p_categoria
    ) LOOP
        DBMS_OUTPUT.PUT_LINE(
            r.NOMBRE || ' - ' || r.PRECIO || ' EUR'
        );
    END LOOP;
END;
/

BEGIN
    mostrar_productos_categoria('Informatica');
END;
/
```

Qué aprender:

- IN
 - FOR
 - recorrer SELECT
-

2. Procedimiento con OUT

Puede caer: contar pedidos

```
CREATE OR REPLACE PROCEDURE contar_pedidos
(p_usuario IN NUMBER,
 p_total OUT NUMBER)
IS
BEGIN
```

```

SELECT COUNT(*)
INTO p_total
FROM PEDIDOS
WHERE ID_USUARIO = p_usuario;
END;
/

DECLARE
  v_total NUMBER;
BEGIN
  contar_pedidos(1, v_total);
  DBMS_OUTPUT.PUT_LINE('Pedidos: ' || v_total);
END;
/

```

Qué aprender:

- **OUT**
- devolver valores

3. Procedimiento con validación

```

CREATE OR REPLACE PROCEDURE actualizar_stock
(p_id IN NUMBER,
 p_cantidad IN NUMBER)
IS
  v_stock NUMBER;
BEGIN
  SELECT STOCK
  INTO v_stock
  FROM PRODUCTOS
  WHERE ID_PRODUCTO = p_id;

  IF v_stock >= p_cantidad THEN
    UPDATE PRODUCTOS
    SET STOCK = STOCK - p_cantidad
    WHERE ID_PRODUCTO = p_id;

    DBMS_OUTPUT.PUT_LINE('Stock actualizado');
  ELSE
    DBMS_OUTPUT.PUT_LINE('Stock insuficiente');
  END IF;
END;
/

```

Repasa:

- IF
 - UPDATE
 - validación
-

4. Cursor explícito

Este cae fijo

```
DECLARE
  CURSOR c_productos IS
    SELECT NOMBRE, PRECIO
    FROM PRODUCTOS;

  v_nombre PRODUCTOS.NOMBRE%TYPE;
  v_precio PRODUCTOS.PRECIO%TYPE;

BEGIN
  OPEN c_productos;

  LOOP
    FETCH c_productos INTO v_nombre, v_precio;
    EXIT WHEN c_productos%NOTFOUND;

    DBMS_OUTPUT.PUT_LINE(
      v_nombre || ' - ' || v_precio
    );
  END LOOP;

  CLOSE c_productos;
END;
/
```

Memoriza:

- OPEN
 - FETCH
 - EXIT WHEN
 - CLOSE
-

5. Cursor con parámetro

```
DECLARE
  CURSOR c_estado(p_estado VARCHAR2) IS
    SELECT ID_PEDIDO
    FROM PEDIDOS
    WHERE ESTADO = p_estado;

  v_id NUMBER;

BEGIN
  OPEN c_estado('ENTREGADO');

  LOOP
    FETCH c_estado INTO v_id;
    EXIT WHEN c_estado%NOTFOUND;

    DBMS_OUTPUT.PUT_LINE('Pedido: ' || v_id);
  END LOOP;

  CLOSE c_estado;
END;
/
```

Repasa:

- parámetros en cursor
-

6. Uso de atributos cursor

```
BEGIN
  UPDATE PRODUCTOS
  SET PRECIO = PRECIO * 1.10
  WHERE CATEGORIA = 'Moviles';

  IF SQL%FOUND THEN
    DBMS_OUTPUT.PUT_LINE(
      'Filas: ' || SQL%ROWCOUNT
    );
  END IF;
END;
/
```

Memoriza:

- %FOUND
 - %NOTFOUND
 - %ROWCOUNT
-

7. Trigger AFTER INSERT

Muy probable

```
CREATE OR REPLACE TRIGGER trg_log_usuario
AFTER INSERT ON USUARIOS
FOR EACH ROW
BEGIN
    DBMS_OUTPUT.PUT_LINE(
        'Usuario insertado: ' || :NEW.NOMBRE
    );
END;
/
```

Repasa:

- AFTER INSERT
 - FOR EACH ROW
 - :NEW
-

8. Trigger BEFORE UPDATE

```
CREATE OR REPLACE TRIGGER trg_control_precio
BEFORE UPDATE OF PRECIO ON PRODUCTOS
FOR EACH ROW
BEGIN
    IF :NEW.PRECIO < 0 THEN
        RAISE_APPLICATION_ERROR(
            -20001,
            'Precio invalido'
        );
    END IF;
END;
/
```

Repasa:

- :OLD
 - :NEW
 - RAISE_APPLICATION_ERROR
-

9. Trigger BEFORE DELETE

```
CREATE OR REPLACE TRIGGER trg_bloquear_borrado
BEFORE DELETE ON USUARIOS
FOR EACH ROW
BEGIN
    RAISE_APPLICATION_ERROR(
        -20002,
        'No se puede borrar'
    );
END;
/
```

10. Excepción clásica

```
DECLARE
    v_nombre PRODUCTOS.NOMBRE%TYPE;
BEGIN
    SELECT NOMBRE
    INTO v_nombre
    FROM PRODUCTOS
    WHERE ID_PRODUCTO = 99;

    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            DBMS_OUTPUT.PUT_LINE('Producto no encontrado');
END;
/
```

Repasa:

- NO_DATA_FOUND
 - WHEN OTHERS
-

Lo que debes sabértelo de memoria

Procedimientos

```
CREATE OR REPLACE PROCEDURE  
IS  
BEGIN  
END;  
/
```

Cursores

```
CURSOR  
OPEN  
FETCH  
EXIT WHEN  
CLOSE
```

Triggers

```
CREATE OR REPLACE TRIGGER  
BEFORE / AFTER  
FOR EACH ROW  
:NEW  
:OLD
```

Excepciones

```
EXCEPTION  
WHEN NO_DATA_FOUND  
WHEN OTHERS
```